Thomas Tusty

Section 1: Edge Impulse Results

https://studio.edgeimpulse.com/public/997977/live

1) The raw inputs used in the Edge Impulse project are the 3 accelerometer axes:

   accX, **accY**, and **accZ**

   These are time-series acceleration signals collected from the sensor. In the impulse setup, they are sampled at 100 Hz using a 2,000 ms window size with a 220 ms window increase/stride.

2) The NN classifier uses **spectral analysis features** extracted from the raw accelerometer inputs accX, accY, and accZ. These features summarize the frequency content and signal energy of the motion data before classification. The training set shown has **350 training windows**, and the classifier is trained using generated spectral features such as **accX RMS**, **accX Peak 1 Height**, **accY Peak 3 Height**, **accX Peak 1 Freq**, and **accY RMS**.
3) The NN classifier outputs the predicted class for the input motion window. In this project, the two classifier outputs are **nominal** and **off**. The model produces a confidence score for each class, then Edge Impulse selects the class with the highest confidence as the prediction. The validation results show both classes being correctly identified with **100% accuracy**, and the confusion matrix includes the two output labels: **nominal** and **off**.
4) The anomaly detector uses selected **spectral features** generated from the accelerometer inputs. Instead of using all generated features, the anomaly detector was trained on **4 selected features**: **accX RMS**, **accX Peak 1 Height**, **accY Spectral Power 2.0 - 5.0 Hz**, and **accZ Spectral Power 2.0 - 5.0 Hz**. These features were used by the K-means anomaly detection block to model normal behavior and produce an anomaly score for new motion windows.

```
Sampling...
Timing: DSP 36 ms, inference 616 us, anomaly 513 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.531250
  off: 0.468750
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 35 ms, inference 614 us, anomaly 523 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.062500
  off: 0.937500
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 33 ms, inference 613 us, anomaly 501 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.937500
  off: 0.062500
Anomaly prediction: 15.888771
Starting inferencing in 2 seconds...
```

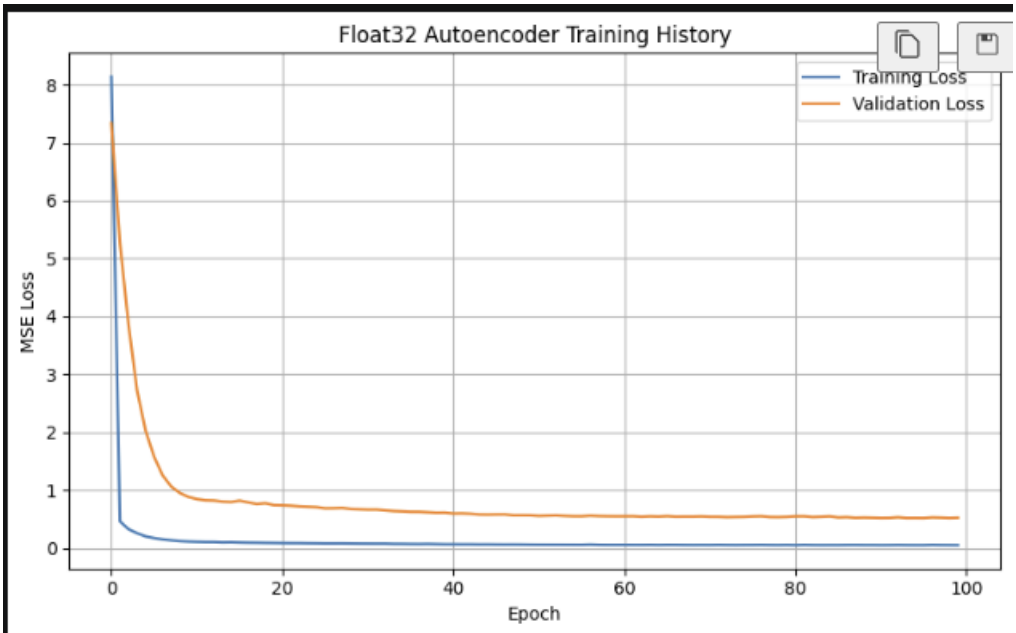
5)

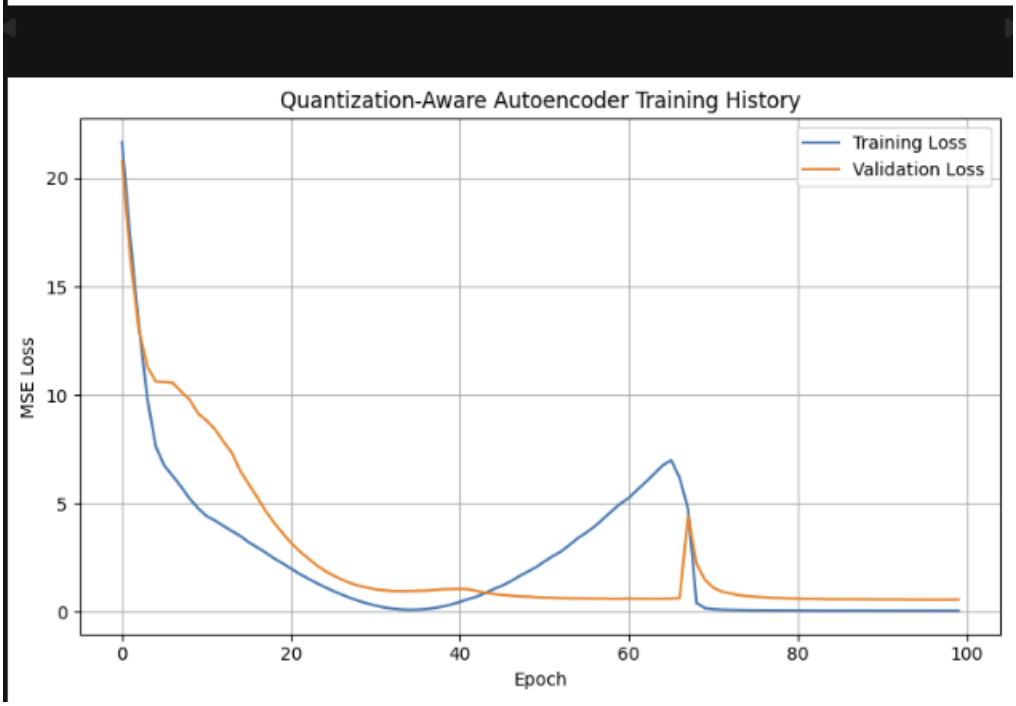
6) The deployment results show that the model is running successfully on the Arduino Nano 33 BLE and producing live classification and anomaly outputs. Most of the later readings are close to off, with values around off: 0.5625 and nominal: 0.4375, but the confidence is not very strong. Some readings are much clearer, such as off: 0.996094 or nominal: 0.937500, which are more trustworthy because one class clearly dominates.

   The classifier output should not always be trusted blindly. It is more reliable when one class has a much higher confidence than the other and when the anomaly score is low or within the expected normal range. It is less reliable when the class probabilities are close together, such as nominal: 0.4375 and off: 0.5625, because the model is basically uncertain. It is also less reliable when the anomaly score is extremely high, like 1000.000000, because that suggests the input looks very different from the training data. In those cases, the classifier may still output a label, but the prediction should be treated with caution.

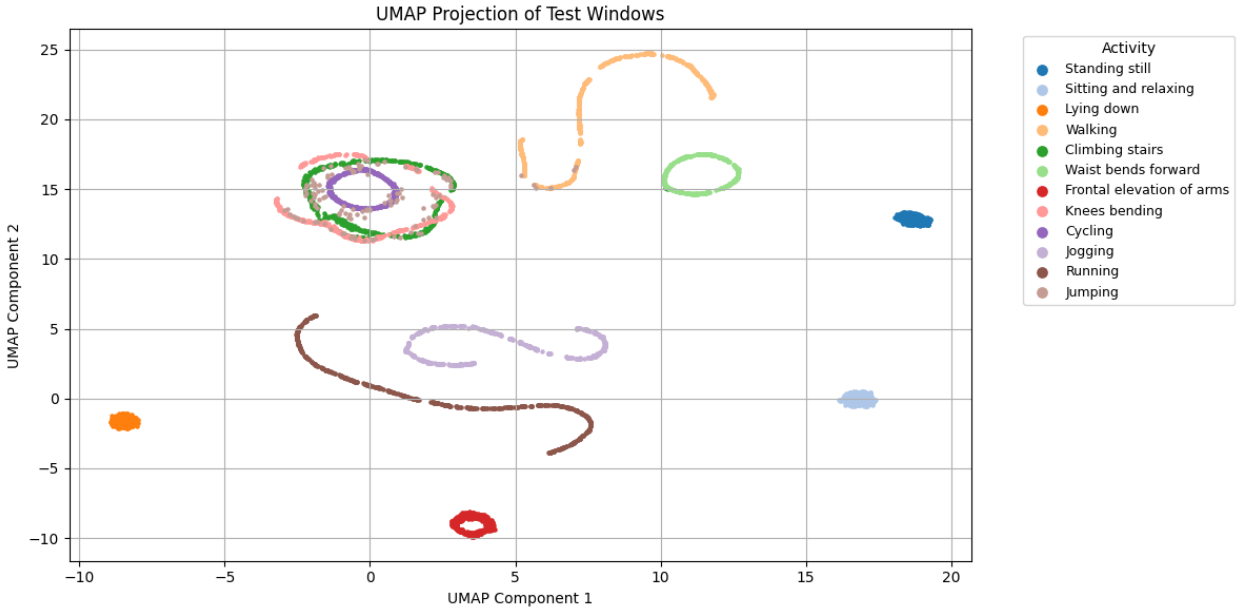
Section 2: Autoencoder Model Results

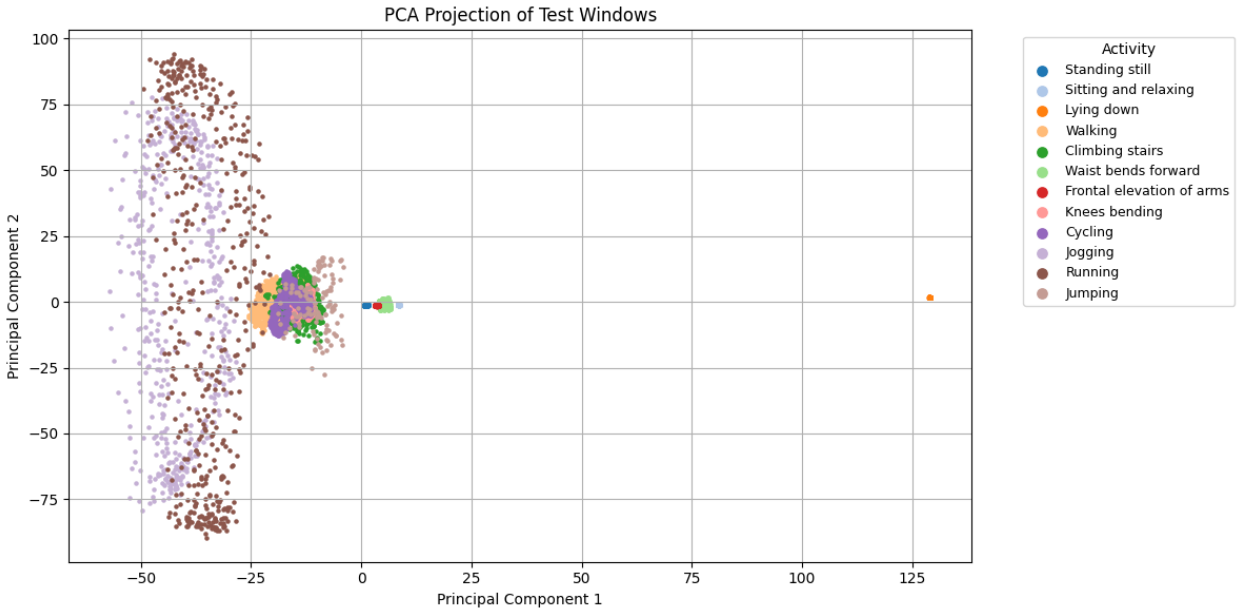
1.

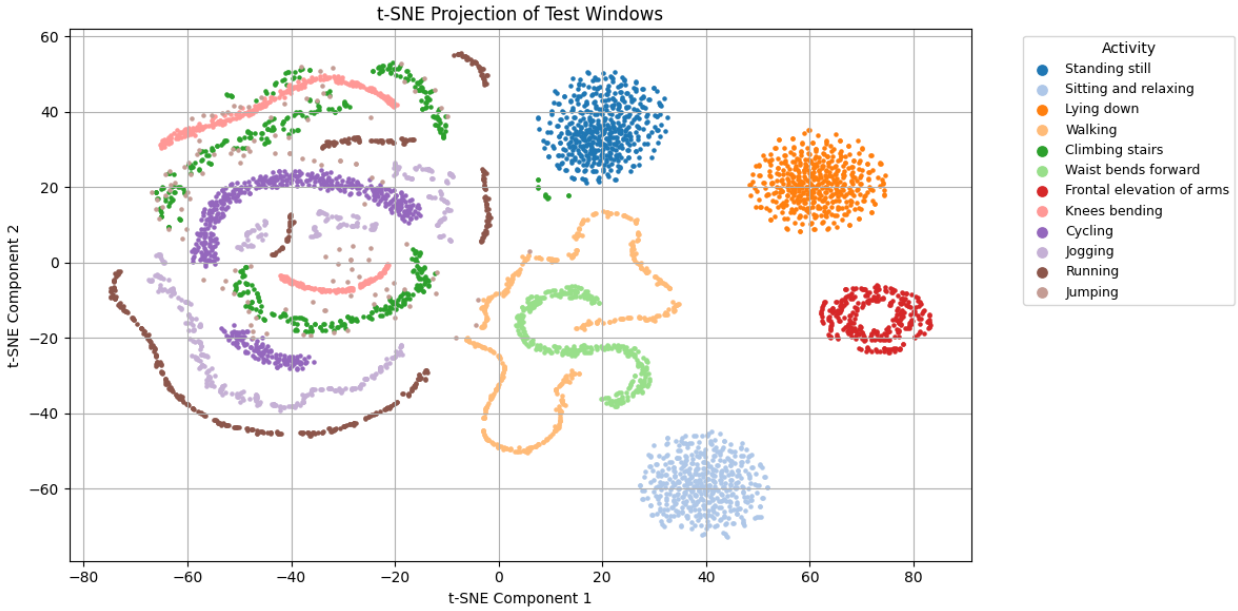




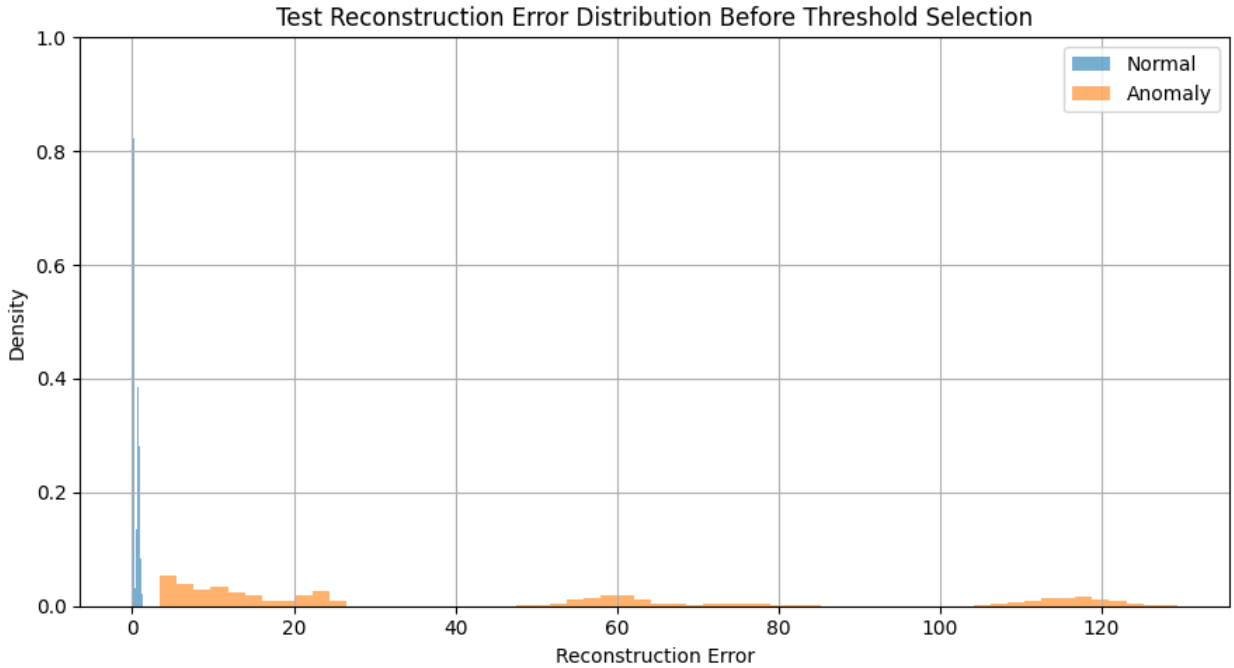
2.

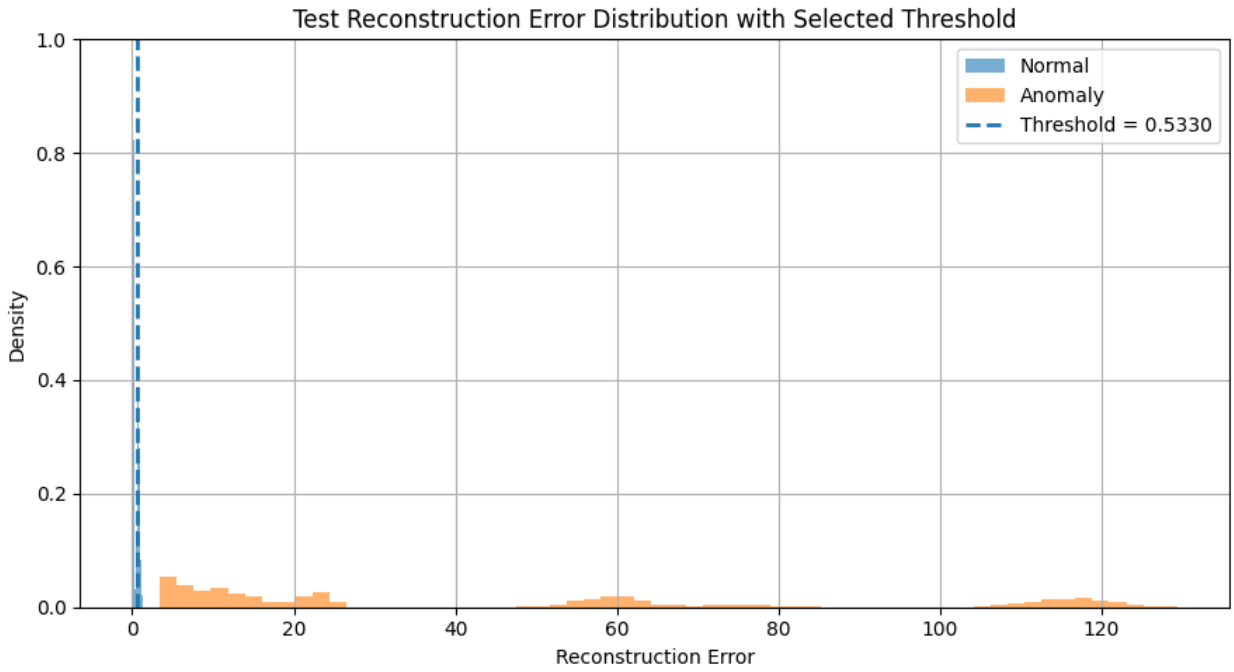

UMAP Projection of Test Windows
UMAP Component 1
UMAP Component 2
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping


PCA Projection of Test Windows
Principal Component 1
Principal Component 2
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping

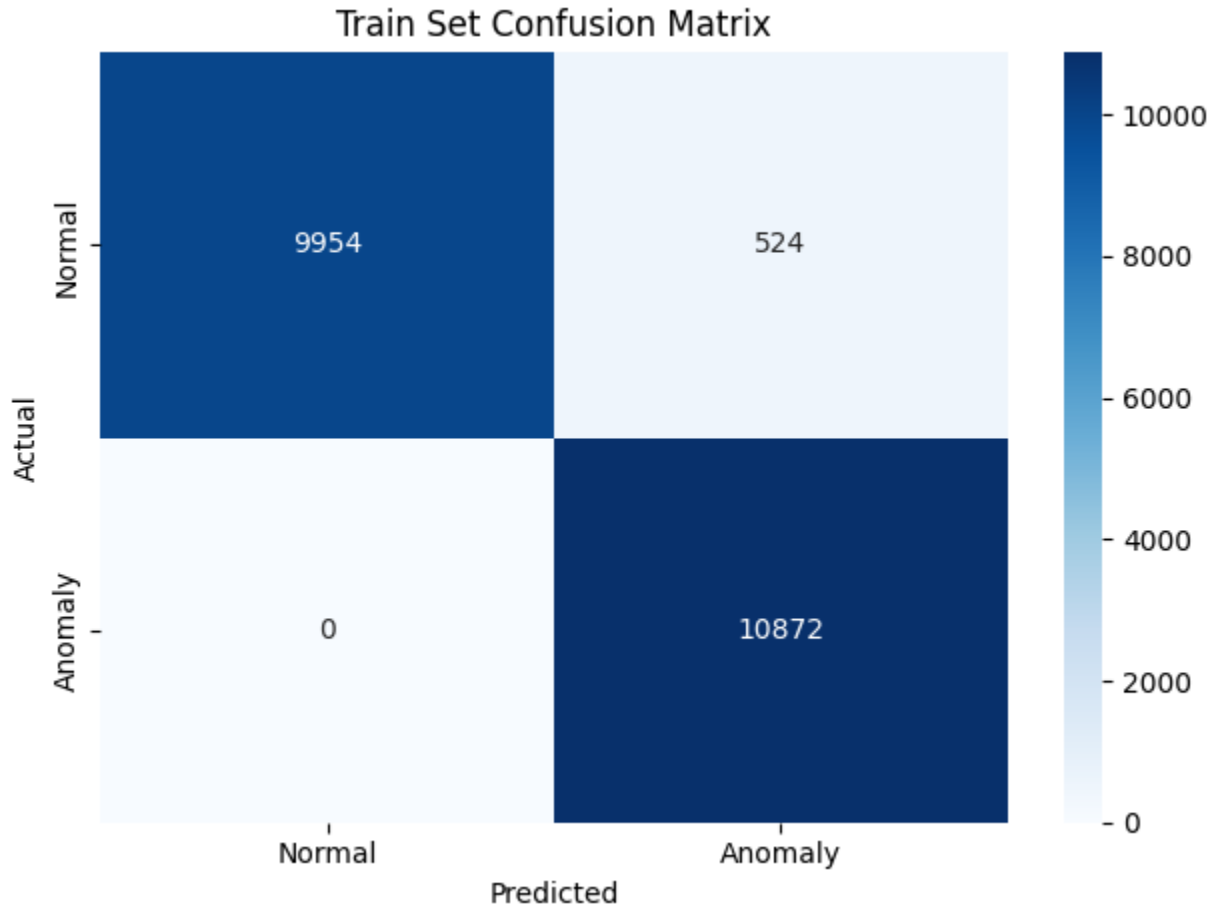

t-SNE Projection of Test Windows
t-SNE Component 2
t-SNE Component 1
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping

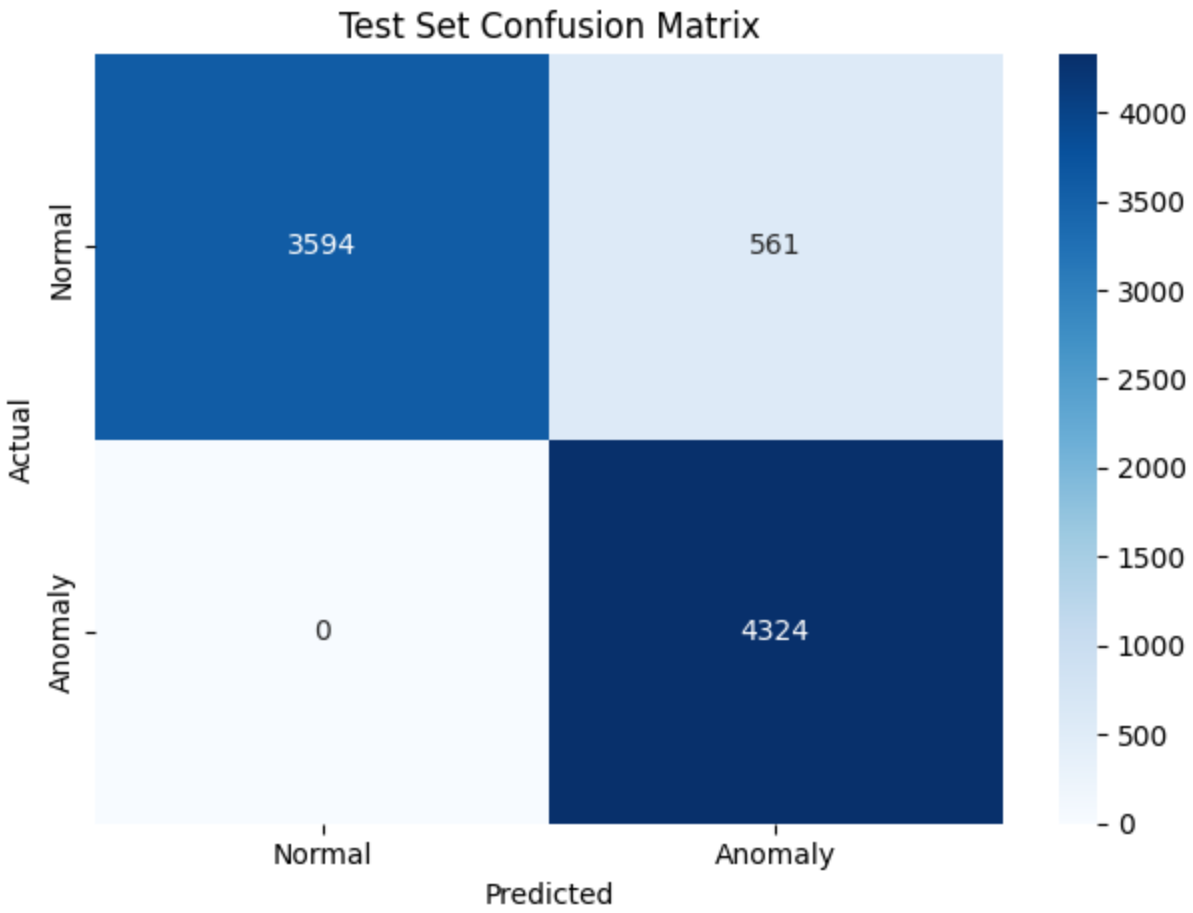
3.


Test Reconstruction Error Distribution Before Threshold Selection
Normal
Anomaly
Density
Reconstruction Error


Test Reconstruction Error Distribution with Selected Threshold
Normal
Anomaly
Threshold = 0.5330
Density
Reconstruction Error
0.0
0.2
0.4
0.6
0.8
1.0
0
20
40
60
80
100
120

4.


Train Set Confusion Matrix
9954
524
0
10872
Normal
Anomaly
Actual
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0



5.

```
23:14:49.749 -> Sample 69/100
23:14:49.749 -> Running inference...
23:14:49.749 -> Reconstruction error: 6.245704
23:14:49.749 -> Result: Anomaly detected
23:14:49.749 ->
```

```
Sample 87/100
Running inference...
Reconstruction error: 0.103319
Result: Normal activity
```

Section 3: Discussion Questions

1) I used a reconstruction error threshold of **0.5330**. This value was chosen from the Python notebook using the normal training reconstruction errors, specifically around the high end of the normal-error distribution. The idea is that normal motion

should usually reconstruct with an error below this value, while unusual or anomalous motion should produce a larger reconstruction error.

2) The autoencoder is trained mostly on normal activity, so it learns how to recreate normal motion patterns well. When the input looks like the normal training data, the reconstructed output should be close to the original input, giving a low reconstruction error. If the input is unusual or different from what the model learned, the autoencoder reconstructs it poorly, which gives a higher reconstruction error. That higher error is used as the signal that the activity may be anomalous.
3) The Arduino deployment needs to preserve the trained TensorFlow Lite model byte array, the model length, the selected reconstruction error threshold, the input size/window shape, and the quantization settings used by the model. It also needs to use the same input feature order, such as accelerometer X, Y, and Z values, and the same windowing assumptions used during training. In the Arduino build, the sketch uses the Nano 33 BLE target and TensorFlow Lite library setup, and the compiled model is included through the generated model file.
4) The model was trained on data prepared in a specific way, so the Arduino must feed it data in the same format. If the notebook used a certain window size, axis order, scaling, flattening order, or normalization method, the Arduino code needs to match that. Otherwise, the model will receive inputs that do not look like the training data, which can cause incorrect reconstruction errors and unreliable normal/anomaly results.
5) The main limitations are memory, processing power, sensor differences, and limited model complexity. A microcontroller has much less RAM and storage than a computer, so the model and preprocessing must stay small. The threshold can also be sensitive to how the device is mounted, how the motion is performed, and whether the real-world data matches the training data. Another limitation is that the autoencoder only knows what “normal” looked like during training, so it may miss subtle anomalies or falsely flag normal behavior if conditions change.